📋 PHARMACY MANAGEMENT SYSTEM COMPLETE DOCUMENTATION

SYSTEM ARCHITECTURE & TECHNICAL SPECIFICATION

📑 TABLE OF CONTENTS

1. EXECUTIVE SUMMARY
2. TECHNOLOGY STACK
3. DATABASE DESIGN & SCHEMA
4. MODULE 1: MEDICINE MANAGEMENT
5. MODULE 2: SUPPLIER MANAGEMENT
6. MODULE 3: BATCH/INVENTORY MANAGEMENT
7. MODULE 4: CUSTOMER MANAGEMENT
8. MODULE 5: SALES & CHECKOUT
9. MODULE 6: DASHBOARD & ANALYTICS
10. MODULE 7: NOTIFICATION SYSTEM
11. MODULE 8: WASTAGE & RETURNS
12. MODULE 9: SUPPLIER ORDERS
13. API REFERENCE
14. DEPLOYMENT GUIDE
15. CONCLUSION & NEXT STEPS

1. EXECUTIVE SUMMARY

Project Overview

The Pharmacy Management System is a comprehensive, enterprise-grade solution designed for local pharmacies in Ethiopia. It provides end-to-end inventory management, sales processing, customer tracking, financial reporting, and automated Telegram-based notifications.

Objective	Solution
Track medicine inventory with expiry dates	Batch-level tracking system
Process sales with prescription validation	Simple phone-based checkout
Alert staff about low stock and expiring medicines	Telegram notification system
Generate financial reports and insights	Real-time dashboard analytics
Manage supplier communications	Telegram-based purchase orders
Track damaged/expired medicine losses	Wastage reporting module

2. TECHNOLOGY STACK

Backend

Technology	Version	Purpose
Node.js	**v22.22.0**	**JavaScript runtime**
Express.js	**v4.18.2**	**Web framework**
TypeScript	**v5.0+**	**Type safety & developer experience**
PostgreSQL	**v14+**	**Primary database**
Sequelize ORM	**v6.37.7**	**Database abstraction & modeling**

```
{
  "dependencies": {
    "express": "^4.18.2",
    "pg": "^8.11.0",
    "pg-hstore": "^2.3.4",
    "sequelize": "^6.32.1",
    "bcryptjs": "^2.4.3",
    "jsonwebtoken": "^9.0.0",
    "dotenv": "^16.0.3",
    "cors": "^2.8.5",
    "helmet": "^7.0.0",
    "morgan": "^1.10.0",
    "express-validator": "^6.15.0",
    "nodemailer": "^6.9.0",
    "node-telegram-bot-api": "^0.64.0",
    "pdfkit": "^0.14.0",
    "node-cron": "^3.0.0"
  },
  "devDependencies": {
    "typescript": "^5.0.0",
    "@types/node": "^20.0.0",
    "@types/express": "^4.17.17",
    "@types/sequelize": "^4.28.14",
    "ts-node": "^10.9.0",
    "nodemon": "^3.0.0"
  }
}
```

System Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐

│                                    CLIENT LAYER
│

│                (Postman, Curl, Telegram,
Frontend)            │

└──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘

                  │

                  ▼
```

┌──┐

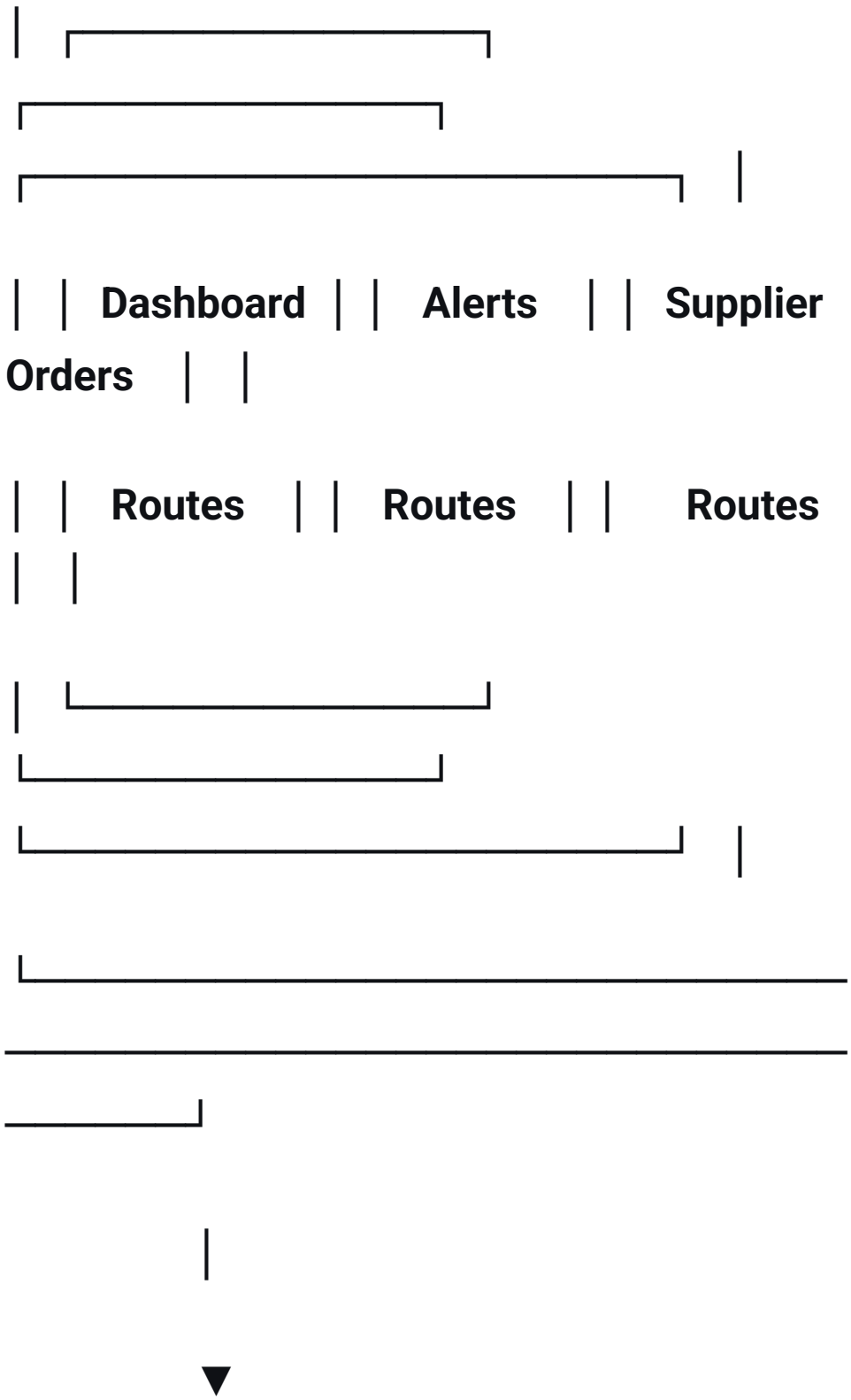
│ **API LAYER (Express)**
│

│ ┌──────────────────────┐
┌──────────────────────┐
┌──────────────────────────────────────┐ │

│ │ **Medicine** │ │ **Batch** │ │
Checkout │ │

│ │ **Routes** │ │ **Routes** │ │ **Routes**
│ │

│ └──────────────────────┘
└──────────────────────┘
└──────────────────────────────────────┘ │


Dashboard
Alerts
Supplier Orders
Routes
Routes
Routes

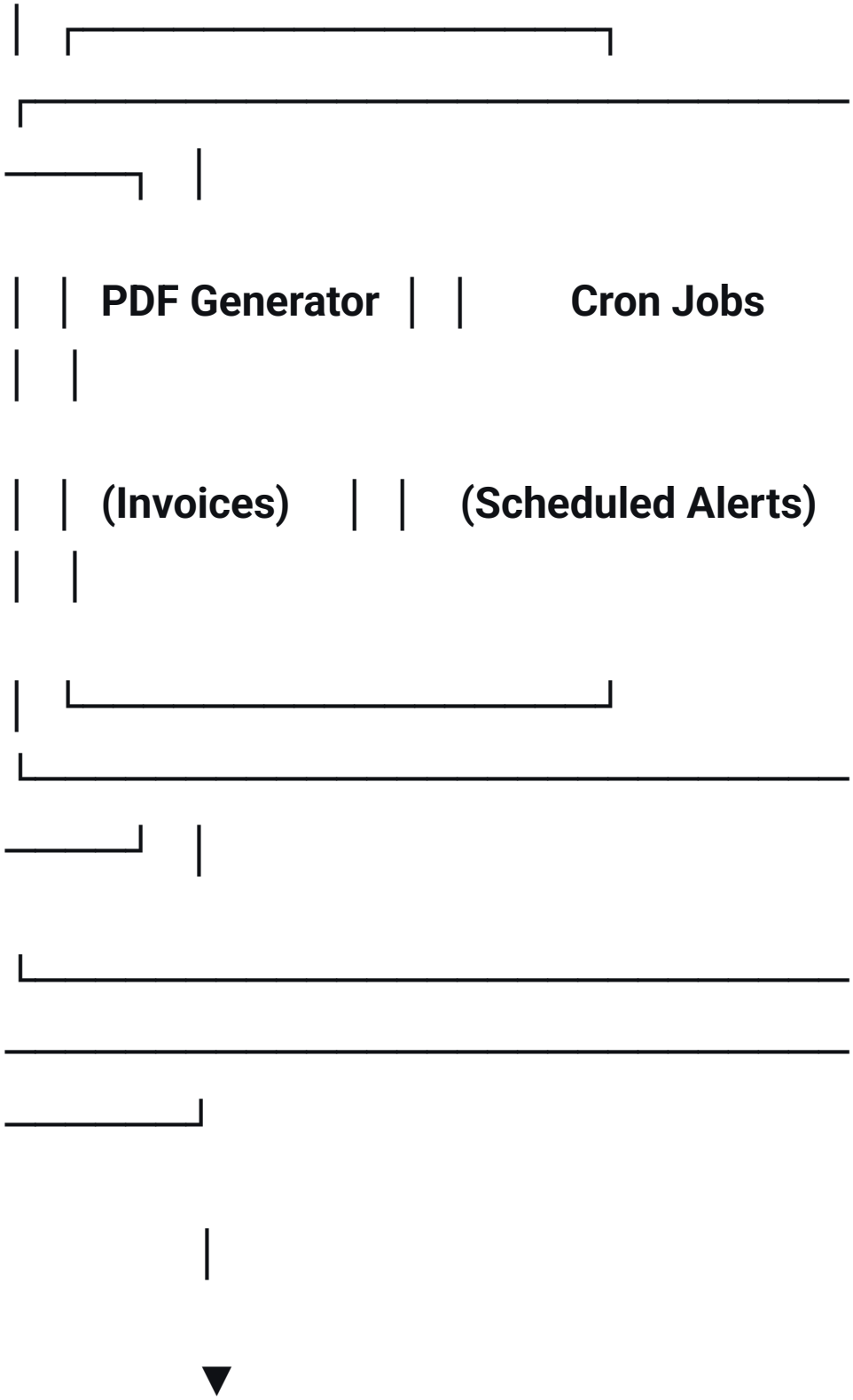
┌──┐

│ SERVICE LAYER (Business Logic) │

│ ┌──────────────────────────────┐ ┌──┐ │

│ │ Notification Service │ │ Telegram Integration (Bot API) │ │

│ └──────────────────────────────┘ └──┘ │


PDF Generator
Cron Jobs
(Invoices)
(Scheduled Alerts)

┌──┐

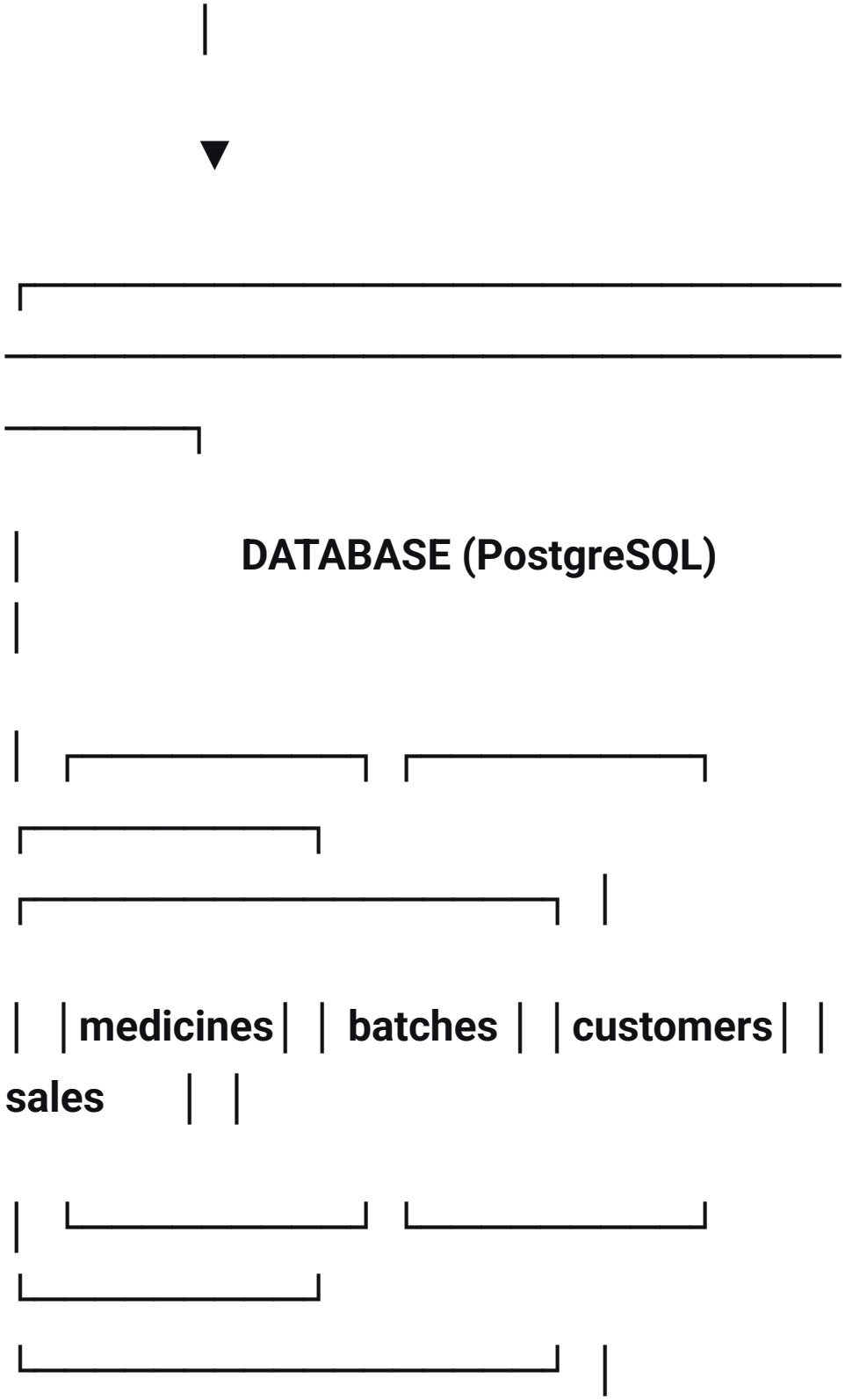
│ **DATA LAYER (Sequelize ORM)**
│

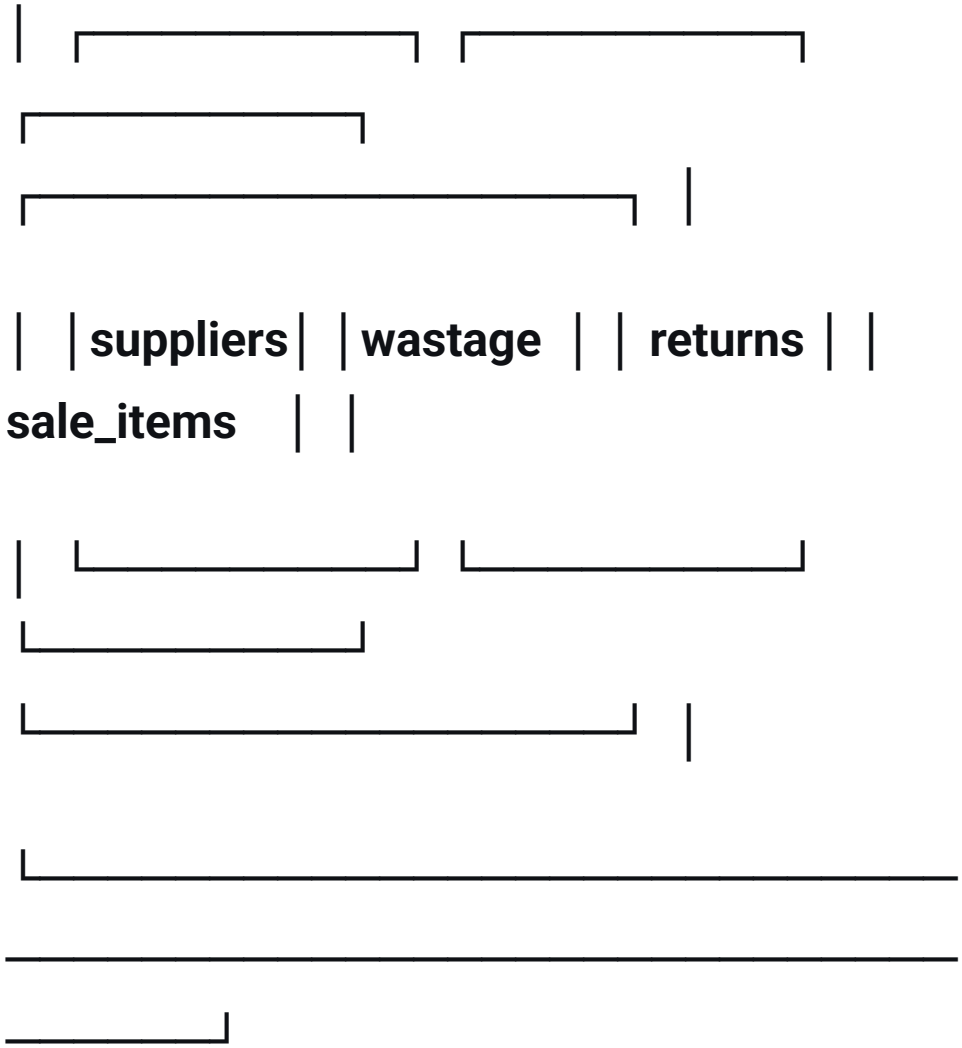
│ ┌──────────────────────┐
┌──────────────────────┐
┌──────────────────────────────────────┐ │

│ │ **Models** │ │ **Migrations** │ │
Seeders │ │

│ └──────────────────────┘
└──────────────────────┘
└──────────────────────────────────────┘ │

└──┘


DATABASE (PostgreSQL)
medicines
batches
customers
sales


suppliers
wastage
returns
sale_items

3. DATABASE DESIGN & SCHEMA

Entity Relationship Diagram (ERD)

```
┌──────────────────────┐      ┌──────────────────────┐
┌──────────────────────┐
│     users      │      │   medicines    │      │   suppliers    │
├──────────────────────┤      ├──────────────────────┤
├──────────────────────┤
│ user_id (PK)   │      │ medicine_id (PK)│      │ supplier_id (PK)│
│ username       │──────────│ name           │      │ name           │
│ password_hash  │      │ generic_name   │      │ contact_person │
│ full_name      │      │ brand          │      │ email          │
│ role           │      │ category       │      │ phone          │
│ is_active      │      │ form           │      │ address        │
│ created_at     │      │ strength       │      │ is_active      │
└──────────────────────┘      │ unit           │      │ created_at     │
                              │ barcode        │      └──────────────────────┘
                              │ reorder_level  │
                              │ is_active      │
                              │ created_at     │
                              │ updated_at     │
                              └──────────────────────┘
                                     │                 │
                                     │                 │
                                     │                 │
                              ┌──────────▼──────────┐
                              │     batches    │◄──────────────────┘
                              ├──────────────────────┤
                              │ batch_id (PK)  │
                              │ batch_number   │
                              │ medicine_id (FK)│
                              │ expiry_date    │
                              │ manufacturing_date│
                              │ supplier_id (FK)│
                              │ quantity       │
                              │ cost_price     │
                              │ selling_price  │
                              │ is_active      │
                              │ created_at     │
                              │ updated_at     │
                              └──────────────────────┘
```

```
                    |
        ┌───────────┼──────────────┐
        |           |              |
┌──────▼──────┐ ┌──────▼──────┐ ┌──────▼──────┐
| sale_items  | | inventory_  | | expiry_alerts |
├─────────────┤ | transactions | ├─────────────┤
| sale_item_id | ├─────────────┤ | alert_id    |
| sale_id (FK) | | transaction_id | | batch_id (FK) |
| batch_id (FK) | | batch_id (FK) | | alert_date  |
| medicine_id  | | transaction_ | | days_until_ |
| medicine_name | |  type     | |  expiry    |
| batch_number | | quantity_  | | is_resolved |
| quantity    | |  change   | | resolved_at |
| unit_price  | | previous_ | | notes       |
| total_price | |  quantity | └─────────────┘
└─────────────┘ | new_quantity |
      |         | reference_id |
      |         | notes        |
      |         | user_id (FK) |
      |         | transaction_ |
      |         |  date        |
      |         └──────────────┘
      |
┌──────▼──────┐     ┌──────────────┐
|   sales     |     |   wastage    |
├─────────────┤     ├──────────────┤
| sale_id (PK) |     | wastage_id (PK) |
| customer_id |     | batch_id (FK) |
| invoice_number |     | medicine_id  |
| customer_name |     | medicine_name |
| customer_phone |     | batch_number |
| prescription_ |     | quantity     |
|  id         |     | cost_price   |
| doctor_name |     | total_loss   |
| items_total |     | reason       |
| discount    |     | notes        |
| tax         |     | reported_by  |
| total_amount |     | reported_date |
| payment_method |     └──────────────┘
| payment_status |
| sale_date   |     ┌──────────────┐
| sold_by (FK) |     |   returns    |
| notes       |     ├──────────────┤
└─────────────┘     | return_id (PK) |
```

```
        │              │ return_type     │
        │              │ reference_id    │
┌──────────▼──────────┐       │ batch_id (FK)   │
│  customers    │       │ medicine_id     │
├──────────────────────┤       │ medicine_name   │
│ customer_id(PK)│      │ batch_number    │
│ name          │       │ quantity        │
│ phone         │       │ unit_price      │
│ address       │       │ total_amount    │
│ total_visits  │       │ reason          │
│ last_visit    │       │ status          │
│ created_at    │       │ notes           │
└──────────────────────┘       │ created_by      │
                 │ approved_by     │
                 │ created_at      │
                 │ updated_at      │
                 └──────────────────────┘
```

```
============================================
-- DATABASE: pharmacy_inventory
-- CREATED: 2026
-- ============================================

-- Enable UUID extension
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- ============================================
-- TABLE: users
```

```sql
-- PURPOSE: Pharmacy staff authentication & roles
-- ==============================================
CREATE TABLE users (
    user_id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    full_name VARCHAR(255) NOT NULL,
    role VARCHAR(50) DEFAULT 'pharmacist',
    is_active BOOLEAN DEFAULT true,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_users_username ON users(username);
CREATE INDEX idx_users_role ON users(role);
```

```sql
-- ==============================================
-- TABLE: medicines
-- PURPOSE: Medicine master catalog
-- ==============================================
CREATE TABLE medicines (
    medicine_id SERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    generic_name VARCHAR(255),
    brand VARCHAR(100),
    category VARCHAR(100),
    form VARCHAR(50),
    strength VARCHAR(100),
    unit VARCHAR(50),
    barcode VARCHAR(50) UNIQUE,
    reorder_level INTEGER DEFAULT 10,
    is_active BOOLEAN DEFAULT true,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE INDEX idx_medicines_name ON medicines(name);
CREATE INDEX idx_medicines_category ON medicines(category);
CREATE INDEX idx_medicines_barcode ON medicines(barcode);

-- ============================================
-- TABLE: suppliers
-- PURPOSE: Medicine vendors/suppliers
-- ============================================
CREATE TABLE suppliers (
    supplier_id SERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL UNIQUE,
    contact_person VARCHAR(100),
    email VARCHAR(100),
    phone VARCHAR(20),
    address TEXT,
    is_active BOOLEAN DEFAULT true,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_suppliers_name ON suppliers(name);
CREATE INDEX idx_suppliers_email ON suppliers(email);

-- ============================================
-- TABLE: batches
-- PURPOSE: Physical stock batches with expiry
-- CORE BUSINESS LOGIC: FIFO/FEFO compliance
-- ============================================
CREATE TABLE batches (
    batch_id SERIAL PRIMARY KEY,
    batch_number VARCHAR(100) NOT NULL,
```

```
    medicine_id INTEGER NOT NULL REFERENCES medicines(medicine_id) ON DELETE
CASCADE,
    expiry_date DATE NOT NULL,
    manufacturing_date DATE,
    supplier_id INTEGER REFERENCES suppliers(supplier_id) ON DELETE SET NULL,
    quantity INTEGER NOT NULL DEFAULT 0 CHECK (quantity >= 0),
    cost_price DECIMAL(10,2),
    selling_price DECIMAL(10,2),
    is_active BOOLEAN DEFAULT true,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_selling_price CHECK (selling_price >= cost_price),
    CONSTRAINT chk_expiry_date CHECK (expiry_date > manufacturing_date)
);

CREATE INDEX idx_batches_batch_number ON batches(batch_number);
CREATE INDEX idx_batches_medicine_id ON batches(medicine_id);
CREATE INDEX idx_batches_expiry_date ON batches(expiry_date);
CREATE INDEX idx_batches_medicine_expiry ON batches(medicine_id, expiry_date);

-- ============================================
-- TABLE: customers
-- PURPOSE: Pharmacy customers/patients
-- BUSINESS RULE: Phone number as primary identifier
-- ============================================
CREATE TABLE customers (
    customer_id SERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    phone VARCHAR(20) UNIQUE NOT NULL,
    address TEXT,
    total_visits INTEGER DEFAULT 0,
    last_visit TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_customers_phone ON customers(phone);
CREATE INDEX idx_customers_name ON customers(name);

-- ============================================
-- TABLE: sales
-- PURPOSE: Sales transactions & invoices
-- ============================================
```

```sql
CREATE TABLE sales (
    sale_id SERIAL PRIMARY KEY,
    invoice_number VARCHAR(50) UNIQUE NOT NULL,
    customer_id INTEGER NOT NULL REFERENCES customers(customer_id),
    customer_name VARCHAR(255) NOT NULL,
    customer_phone VARCHAR(20) NOT NULL,
    prescription_id VARCHAR(100) NOT NULL,
    doctor_name VARCHAR(100),
    items_total DECIMAL(10,2) NOT NULL,
    discount DECIMAL(10,2) DEFAULT 0,
    tax DECIMAL(10,2) DEFAULT 0,
    total_amount DECIMAL(10,2) NOT NULL,
    payment_method VARCHAR(50) NOT NULL,
    payment_status VARCHAR(20) DEFAULT 'paid',
    sale_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    sold_by INTEGER NOT NULL REFERENCES users(user_id),
    notes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_sales_invoice ON sales(invoice_number);
CREATE INDEX idx_sales_customer ON sales(customer_id);
CREATE INDEX idx_sales_date ON sales(sale_date);

-- ============================================
-- TABLE: sale_items
-- PURPOSE: Individual line items in sales
-- ============================================
CREATE TABLE sale_items (
    sale_item_id SERIAL PRIMARY KEY,
    sale_id INTEGER NOT NULL REFERENCES sales(sale_id) ON DELETE CASCADE,
    batch_id INTEGER NOT NULL REFERENCES batches(batch_id),
    medicine_id INTEGER NOT NULL,
    medicine_name VARCHAR(255) NOT NULL,
    batch_number VARCHAR(100) NOT NULL,
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    unit_price DECIMAL(10,2) NOT NULL,
    total_price DECIMAL(10,2) NOT NULL
);
```

```
CREATE INDEX idx_sale_items_sale ON sale_items(sale_id);
CREATE INDEX idx_sale_items_batch ON sale_items(batch_id);

-- ============================================
-- TABLE: wastage
-- PURPOSE: Track damaged, expired, lost medicines
-- BUSINESS VALUE: Loss analysis & prevention
-- ============================================
CREATE TABLE wastage (
    wastage_id SERIAL PRIMARY KEY,
    batch_id INTEGER NOT NULL REFERENCES batches(batch_id),
    medicine_id INTEGER NOT NULL,
    medicine_name VARCHAR(255) NOT NULL,
    batch_number VARCHAR(100) NOT NULL,
    quantity INTEGER NOT NULL,
    cost_price DECIMAL(10,2) NOT NULL,
    total_loss DECIMAL(10,2) NOT NULL,
    reason VARCHAR(50) NOT NULL,
    notes TEXT,
    reported_by INTEGER NOT NULL REFERENCES users(user_id),
    reported_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_wastage_batch ON wastage(batch_id);
CREATE INDEX idx_wastage_date ON wastage(reported_date);

-- ============================================
-- TABLE: returns
-- PURPOSE: Customer returns & supplier returns
-- ============================================
CREATE TABLE returns (
    return_id SERIAL PRIMARY KEY,
    return_type VARCHAR(20) NOT NULL,
    reference_id VARCHAR(50) NOT NULL,
    batch_id INTEGER NOT NULL REFERENCES batches(batch_id),
    medicine_id INTEGER NOT NULL,
    medicine_name VARCHAR(255) NOT NULL,
    batch_number VARCHAR(100) NOT NULL,
    quantity INTEGER NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL,
    total_amount DECIMAL(10,2) NOT NULL,
    reason VARCHAR(50) NOT NULL,
    status VARCHAR(20) DEFAULT 'PENDING',
    notes TEXT,
```

```
    created_by INTEGER NOT NULL REFERENCES users(user_id),
    approved_by INTEGER REFERENCES users(user_id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_returns_batch ON returns(batch_id);
CREATE INDEX idx_returns_status ON returns(status);
CREATE INDEX idx_returns_date ON returns(created_at);

-- ============================================
-- TABLE: inventory_transactions
-- PURPOSE: Complete audit trail for all stock movements
-- ============================================
CREATE TABLE inventory_transactions (
    transaction_id SERIAL PRIMARY KEY,
    batch_id INTEGER NOT NULL REFERENCES batches(batch_id),
    transaction_type VARCHAR(20) NOT NULL,
    quantity_change INTEGER NOT NULL,
    previous_quantity INTEGER NOT NULL,
    new_quantity INTEGER NOT NULL,
    reference_id VARCHAR(100),
    notes TEXT,
    performed_by INTEGER REFERENCES users(user_id),
    transaction_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_inv_trans_batch ON inventory_transactions(batch_id);
CREATE INDEX idx_inv_trans_date ON inventory_transactions(transaction_date);
CREATE INDEX idx_inv_trans_type ON inventory_transactions(transaction_type);
```

Database Constraints & Business Rules

Constraint	Table	Purpose
`CHECK (quantity >= 0)`	batches	Prevent negative inventory

`CHECK (selling_price >= cost_price)`	batches	Ensure profit margin
`CHECK (expiry_date > manufacturing_date)`	batches	Valid date logic
`UNIQUE (barcode)`	medicines	Prevent duplicate products
`UNIQUE (phone)`	customers	Phone as primary identifier
`UNIQUE (invoice_number)`	sales	Prevent duplicate invoices
`FOREIGN KEY (medicine_id)`	batches	Referential integrity
`DEFAULT (is_active = true)`	multiple	Soft delete pattern

4. MODULE 1: MEDICINE MANAGEMENT

Business Requirements

- Maintain master catalog of all medicines
- Track by generic name, brand, category, form, strength
- Unique barcode identification
- Set reorder levels for automatic alerts
- Soft delete (deactivate) instead of hard delete

Data Model Analysis

```typescript
```

```typescript
interface Medicine {
  medicine_id: number;        // Primary key, auto-increment
  name: string;              // Required, indexed
  generic_name: string;      // Scientific name
  brand: string;            // Brand name
  category: string;         // Therapeutic category
  form: string;            // Tablet, Capsule, Syrup, etc.
  strength: string;        // 500mg, 10ml, etc.
  unit: string;           // mg, ml, IU, etc.
  barcode: string;        // Unique, 8-13 digits
  reorder_level: number;  // Alert threshold, default 10
  is_active: boolean;    // Soft delete flag
  created_at: Date;      // Timestamp
  updated_at: Date;      // Timestamp

}
```

API Endpoints Deep Dive

GET /api/medicines

Purpose: Retrieve paginated list of active medicines with current stock levels

Business Logic:

1. Filter `is_active = true` only
2. Calculate total stock from all active batches
3. Determine low stock status (total ≤ reorder_level)
4. Sort alphabetically by name

Performance Optimization:

- Parallel Promise.all for stock calculations
- Batches loaded per medicine individually

Response Structure:

```json
{
  "success": true,
  "count": 25,
  "data": [
    {
      "medicine_id": 1,
      "name": "Paracetamol 500mg",
      "generic_name": "Acetaminophen",
      "total_stock": 450,
      "is_low_stock": false,
      "batches": [...]
    }
  ]
}
```

POST /api/medicines

Purpose: Create new medicine entry

Validation Rules:

- `name`: Required, max 255 chars, unique
- `barcode`: Optional, must be 8-13 digits if provided, unique
- `reorder_level`: Optional, default 10, min 0

Business Rules:

- Prevent duplicate names
- Prevent duplicate barcodes
- Auto-set `is_active = true`

PUT /api/medicines/:id

Purpose: Update existing medicine

Constraints:

- Cannot reactivate deactivated medicine (must create new)
- Barcode uniqueness enforced on update
- Partial updates supported

DELETE /api/medicines/:id

Purpose: Soft delete medicine

Business Rules:

- Check for existing stock before deletion
- Set `is_active = false` only
- Historical sales remain intact (referential integrity)

Complex Queries

Low Stock Detection

```typescript
const totalStock = await Batch.sum('quantity', {
  where: { medicine_id, is_active: true }
});

const isLowStock = totalStock <= medicine.reorder_level;
```

5. MODULE 2: SUPPLIER MANAGEMENT

Business Requirements

- Maintain vendor/supplier database
- Track contact information and payment terms
- Associate suppliers with medicine batches
- Performance rating system

Data Model Analysis

```typescript
interface Supplier {
  supplier_id: number;
  name: string;              // Required, unique
  contact_person: string;    // Primary contact
  email: string;            // Business email
  phone: string;           // Contact number
  address: string;        // Physical location
  payment_terms: string;  // COD, Net 15, Net 30, etc.
  rating: number;        // 1-5 star rating
  is_active: boolean;   // Soft delete

}
```

API Endpoints

Method	Endpoint	Description
GET	`/api/suppliers`	List all active suppliers
POST	`/api/suppliers`	Create new supplier
GET	`/api/suppliers/:id`	Get supplier details
PUT	`/api/suppliers/:id`	Update supplier
DELETE	`/api/suppliers/:id`	Soft delete supplier

Business Logic

- Suppliers cannot be deleted if associated with active batches
- Payment terms standardization (ENUM)
- Automatic rating calculation based on order history

6. MODULE 3: BATCH/INVENTORY MANAGEMENT

Core Business Logic - EXPIRY TRACKING

This is the MOST CRITICAL module for pharmacy operations. Every medicine batch must be tracked independently due to:

1. Different expiry dates - Cannot mix batches
2. Different cost prices - Affects profit calculation
3. FIFO/FEFO compliance - Sell soonest-expiring first
4. Recall capability - Identify affected patients

5. Regulatory compliance - Pharmacy laws require batch tracing

Data Model Analysis

```typescript
interface Batch {
  batch_id: number;
  batch_number: string;     // Manufacturer's batch #
  medicine_id: number;      // Foreign key
  expiry_date: Date;       // Must be future
  manufacturing_date: Date; // Optional but validated
  supplier_id: number;     // Who supplied this batch
  quantity: number;       // Current stock (never negative)
  cost_price: number;    // Purchase price
  selling_price: number; // Retail price (≥ cost)
  is_active: boolean;   // Soft delete

}
```

Instance Methods

`getDaysUntilExpiry()`

```typescript
getDaysUntilExpiry(): number {
  const today = new Date();
  const expiry = new Date(this.expiry_date);
  const diffTime = expiry.getTime() - today.getTime();
  return Math.ceil(diffTime / (1000 * 60 * 60 * 24));
}
```

`getExpiryStatus()`

```typescript
getExpiryStatus(): 'EXPIRED' | 'CRITICAL' | 'WARNING' | 'OK' {
  const days = this.getDaysUntilExpiry();
  if (days < 0) return 'EXPIRED';
  if (days <= 30) return 'CRITICAL';
  if (days <= 90) return 'WARNING';
  return 'OK';
}
```

`canSell(quantity: number): boolean`

```typescript
canSell(quantity: number): boolean {
  return this.is_active &&
         this.quantity >= quantity &&
         this.getDaysUntilExpiry() > 0;
```

```
}
```

Expiry Status Matrix

Status	Days Until Expiry	Color	Action Required
EXPIRED	**< 0**	🔴	**Remove from stock, write off**
CRITICAL	**0-30**	🟠	**Urgent sale, discount, return**
WARNING	**31-90**	🟡	**Monitor, plan markdown**
OK	**> 90**	🟢	**Normal inventory**

API Endpoints Deep Dive

POST /api/batches

Purpose: Add new stock batch

Validation Rules:

- `expiry_date` must be in the future
- `manufacturing_date` < `expiry_date`
- `selling_price` ≥ `cost_price`
- `batch_number` should be unique

Transaction Flow:

1. Verify medicine exists
2. Verify supplier exists (if provided)
3. Validate dates
4. Create batch record

5. Auto-create inventory transaction

PATCH /api/batches/:id/stock

Purpose: Adjust stock quantity (add/remove)

Business Rules:

- `operation: 'add'` - Increase stock
- `operation: 'remove'` - Decrease stock (requires reason)
- Cannot remove more than available
- Transaction logged for audit

GET /api/batches/expiring

Purpose: Identify batches nearing expiry

Query Parameters:

- `days`: Lookahead period (default: 30)
- Returns batches with `expiry_status: 'CRITICAL' | 'WARNING'`

Inventory Transaction Audit Trail

Every stock movement creates an `inventory_transactions` record:

```typescript
{
  transaction_id: 12345,
  batch_id: 1,
  transaction_type: 'SALE' | 'PURCHASE' | 'ADJUSTMENT' | 'RETURN' | 'WASTAGE',
  quantity_change: -5,  // Negative for reductions
  previous_quantity: 100,
  new_quantity: 95,
  reference_id: 'INV-2026-001',  // Link to sale or purchase
  performed_by: 1,  // User ID
  transaction_date: '2026-02-12T10:58:50.315Z'
}
```

This provides complete traceability for:

- Regulatory audits
- Theft investigation
- Process improvement
- Financial reconciliation

7. MODULE 4: CUSTOMER MANAGEMENT

Business Requirements - SIMPLE BY DESIGN

- No complex registration forms
- Phone number as primary identifier
- Auto-create on first purchase
- Track visit frequency and spending

Data Model Analysis

```
interface Customer {
  customer_id: number;
  name: string;           // Required
  phone: string;         // Required, unique
  address?: string;      // Optional
  total_visits: number; // Auto-incremented
  last_visit: Date;     // Auto-updated
```

```
  created_at: Date;
}
```

Key Business Logic - Find or Create Pattern

```
// Called during checkout
let [customer] = await Customer.findOrCreate({
  where: { phone },
  defaults: {
    name,
    phone,
    total_visits: 0,
    last_visit: new Date()
  }
});


// Update visit statistics
await customer.update({
  name,  // Update in case they changed it
  last_visit: new Date(),
  total_visits: sequelize.literal('total_visits + 1')
});
```

Customer Intelligence

The system automatically tracks:

- Lifetime value - Total spending across all visits
- Frequency - Number of visits
- Recency - Last visit date
- Prescription history - Linked to sales

This enables:

- Loyal customer identification
- Refill reminders (future feature)
- Targeted promotions

8. MODULE 5: SALES & CHECKOUT

Business Requirements - FAST & PRACTICAL

- Pharmacist enters customer phone + name
- Prescription ID is mandatory (Ethiopia regulation)
- Select medicines by batch
- Auto-calculate totals
- Apply discount/tax manually
- Payment: Cash or Transfer
- Auto-generate invoice number
- Auto-reduce stock
- Send Telegram notification

Complete Checkout Flow Diagram